Project#3: Pacman Competition

1. Project Overview

- In this project, you will implement **your own agent** for the competition version of Pacman, using any method (e.g., RL, Q-learning, Reflex, etc.)
- This assignment reuses **UC Berkeley's Pacman AI project** (https://ai.berkeley.edu/), but with minimum adjustments for CAU's CSE 17182.

2. Environment

- Projects in this class require Python 3.6+.
- For the grading, I will use vanilla Python 3.9 in a `conda` environment. If you want the same setup, install `conda` and:

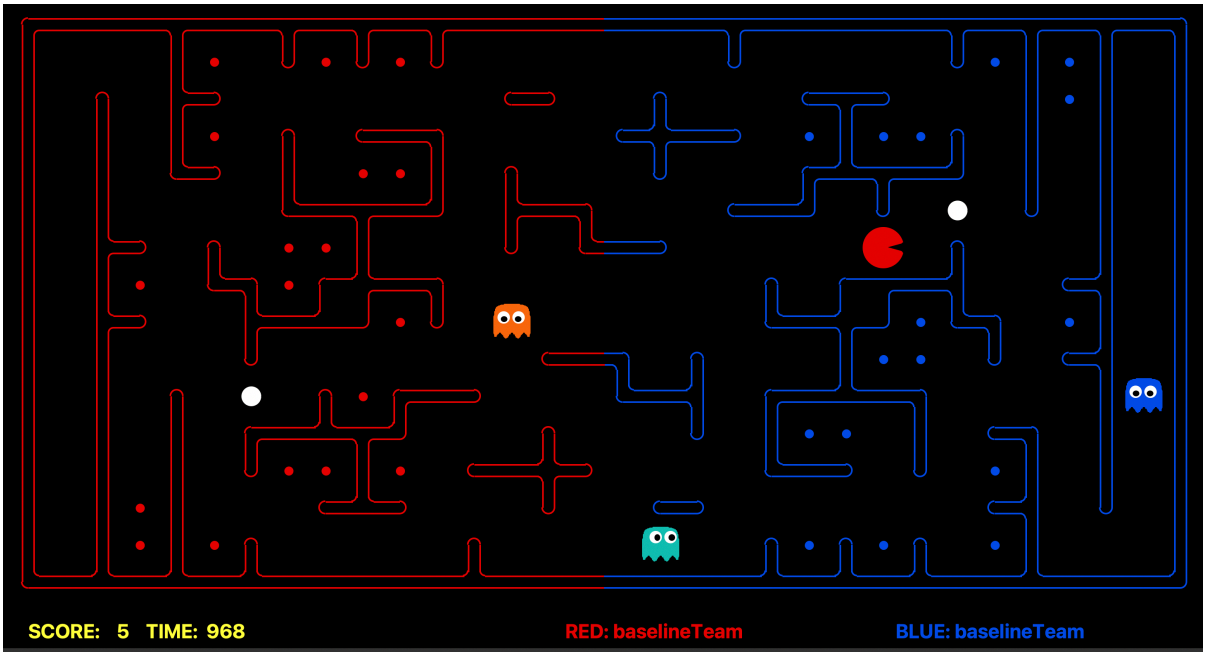
```
conda create -n pacman python=3.9 -y
conda activate pacman
python --version   # should show 3.9.x
```

- Only NumPy (`numpy`) and Python standard libraries are allowed. Any other external libraries are not permitted.

3. **Submission Deliverable & Deadline**

- Deadline: June 3rd (Wed) 23:59 (through the e-class system)
- Within the code files from the given zip archive (project3.zip), you are allowed to submit ONLY `myTeam.py` file with no changed filename.
- Note that the evaluation system does NOT support any separate save/load files. Therefore, if your agent learns any weights, Q-tables, or other parameters during development/training, you must manually include those learned values directly inside `myTeam.py`. No additional data/model files are allowed.
- The submitted `myTeam.py` file size must not exceed 10MB.
- If you violate this deliverable rule, you will be fully responsible for any case in which your agent does not behave as intended under our evaluation setting.

4. Play Competitive Pacman!



- You can simulate Competitive Pacman games between baseline agents as follows:
  - `python capture.py --frameTime 0.05`
    (Red: baselineTeam, Blue: baselineTeam, layout: defaultCapture)

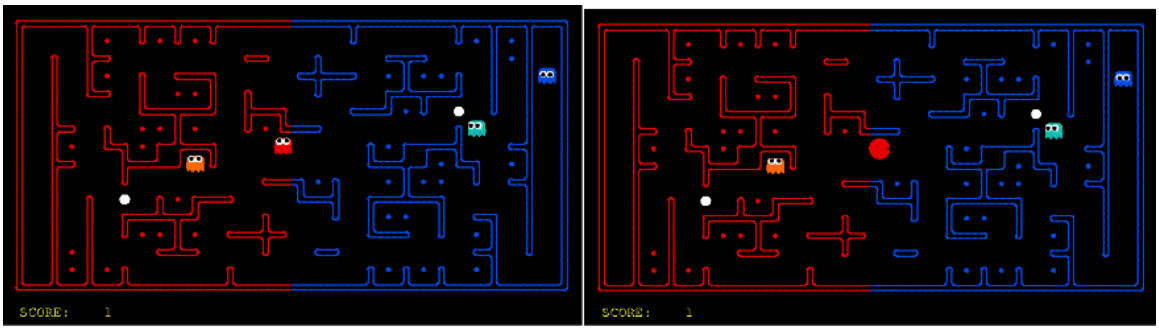
- Or, you can run your own built agent as follows:
  - `python capture.py -r myTeam`
    (Red: myTeam, Blue: baselineTeam, layout: defaultCapture)

- You can configure various settings using command-line arguments:
  - Set teams: `'-r' or '-b'` (e.g., `-r myTeam, -b baselineTeam`)
  - Another layout: `'-l'` (e.g., `-l alleyCapture`)
  - Multiple games: `'-n'` (e.g., `-n 5`)
  - Replay: `'--replay'` (e.g., `--replay replay/TeamA_vs_TeamB.replay`)

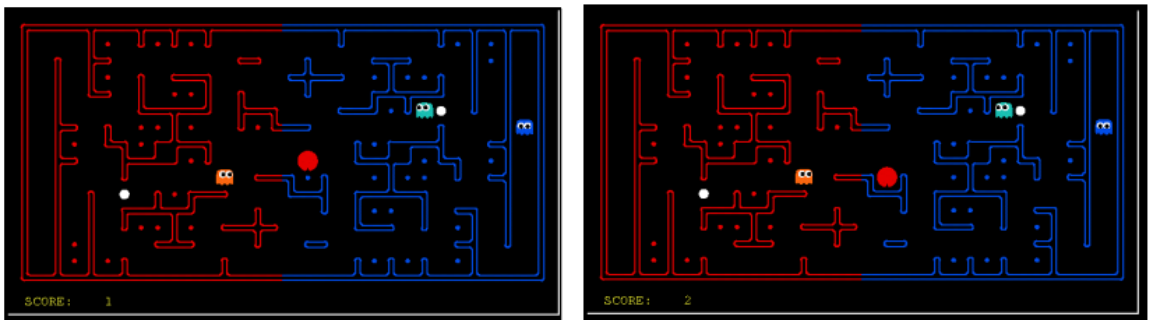
- Please explore the codebase to find additional options.

5. Rules

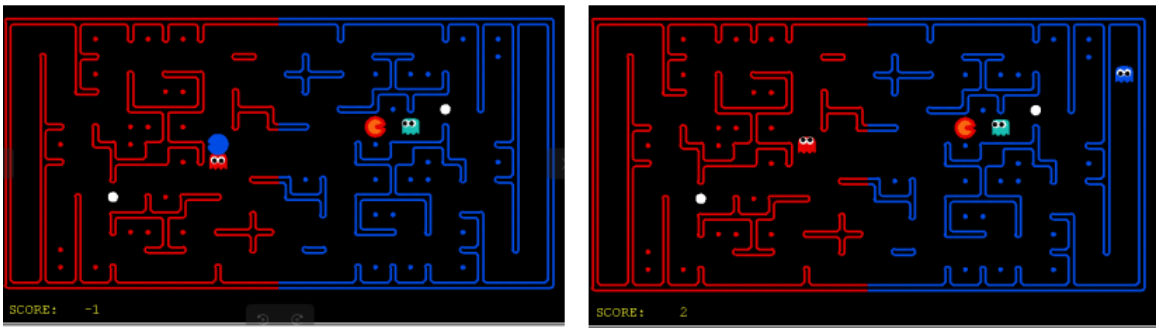
- **Layout:** The Pacman map is divided into two halves: blue (right) and red (left). Red agents must defend the red food while trying to eat the blue food. **When on the red side, a red agent is a ghost. When crossing into enemy territory, the agent becomes a Pacman.**



- **Scoring:** When a Pacman eats a food dot, it earns **one point** per food pellet. Red team scores are positive, while Blue team scores are negative.



- **Eating Pacman:** When a Pacman is eaten by an opposing ghost, the Pacman returns to its starting position (as a ghost). **Three points** are awarded for eating an opponent.



- **Power capsules:** If Pacman eats a power capsule, agents on the opposing team become "scared" for the next 40 moves, or until they are eaten and respawn, whichever comes sooner. Agents that are "scared" are susceptible while in the form of ghosts (i.e. while on their own team's side) to being eaten by Pacman. Specifically, if Pacman collides with a "scared" ghost, Pacman can eat the ghost (and earn **three points**), and the ghost respawns at its starting position (no longer in the "scared" state).

- **Turn Order:** In each game, agents act in the following fixed order: Red Agent 1 → Blue Agent 1 → Red Agent 2 → Blue Agent 2. Therefore, the Red team always moves slightly earlier than the Blue team.

- **Observations:** Agents can only observe an opponent's configuration (position and direction) if they or their teammate is **within 5 squares** (Manhattan distance). Otherwise, an agent always gets a **noisy distance** reading for each agent on the board, which can be used to approximately locate unobserved opponents.

- **Winning:** A game ends when one team eats all but two of the opponents' dots. Games are also limited to 3000 agent moves (750 moves per each of the four agents). If this move limit is reached, whichever team has eaten the most food wins. If the score is zero (i.e., tied) this is recorded as a tie game.

- **Computation Time:** Each agent has **a maximum of 1 second** to return each action. Each move which does not return within one second will incur a warning. **After three warnings, or any single move taking more than 3 seconds, the game is forfeit.** There will be an initial start-up allowance of 5 seconds (you can use the time for running the `'registerInitialState'` function).

- Important Note: The computation time heavily depends on hardware performance. The evaluation of this project will be conducted using an `AMD Ryzen Threadripper 7980X (3.2GHz) CPU.`

6. Competition Rules

1) Minimum Qualification

- To qualify for the main competition, your agent must achieve at least a 65% win rate against the provided `baselineTeam`.
- For evaluation, a total of 20 games will be played against `baselineTeam`:
  - 10 games as the Red team
  - 10 games as the Blue team
- All qualification matches will be played on the `defaultCapture` layout.
- **Only teams that pass the minimum qualification advance to the Group Stage**.
- We provide the auto-grading system, so that you can evaluate your agent using the following command (if you remove the `-q` option, you can watch the rendered gameplay):
  - `python autograder.py -q`
  - The autograder also reports your agent's computation time at each step (both average and maximum), and records all 20 games as replay files, which can be replayed using `capture.py` with the `--record` option.

2) Group Stage

- All qualified teams **(combined across both Class 03 and Class 04)** will participate in the Group Stage.
- A total of **16 groups** will be created.
- Within each group, every pair of teams will play against each other twice:
  - One game with Team A as Red and Team B as Blue
  - One game with Team B as Red and Team A as Blue
- All Group Stage matches will be played on the `defaultCapture` layout.
- Match points are assigned as follows:
  - Win: 3 match points
  - Tie: 1 match point
  - Loss: 0 match points
- After all league matches are completed, **only the top 2 teams** from each group will advance to **the Knockout Stage** (Round of 32).
- Tie-Breaking Rules - If two or more teams have the same number of match points, ranking will be determined in the following order:
  - Head-to-head winner
  - Total achieved score difference
  - Average action computation time (The lower the better)

3) Knockout Stage

- The Knockout Stage will consist of a 32-team single-elimination tournament.
- Each group winner will be matched against a runner-up from another group in the Round of 32.
- Each knockout matchup consists of 4 games:
  - `defaultCapture` layout (1 game as Red / 1 game as Blue)
  - Hidden layout containing additional interesting elements (1 game as Red / 1 game as Blue)
- Tie-Breaking Rules - If the matchup result is tied after 4 games, ranking will be determined in the following order:
  - Total achieved score difference
  - Average action computation time (The lower the better)

7. Grading (Total: 11 points)

- Passing the Minimum Qualification (advancing to the Group Stage): 3 points
- Advancing to the Round of 32: 3 points
- Advancing to the Round of 16: 1 point
- Advancing to the Quarterfinals (Top 8): 1 point
- Advancing to the Semifinals (Top 4): 1 point
- Advancing to the Finals (Top 2): 1 point
- Winning the Tournament (Champion): 1 point

If you have any questions about the project code (or if you find any bugs or unexpected behavior), **please contact the TAs via email:**
최원용 (ihaveconcept@cau.ac.kr), 박은효 (peh4622@cau.ac.kr)

Plagiarism in code will result in an automatic F grade, regardless of the reason. CAU has the internal system of detecting possible reuse of code. You may consult tools like ChatGPT for guidance, but do not copy or submit generated content as-is. This may lead to high similarity across submissions, which will be treated as plagiarism.